

High Performance Programming

Lecture 9:

“Testing and verification”

Lecturer: Ramoni Adeogun



AALBORG
UNIVERSITET

Teaching plan

Parallel concepts, models and algorithms

- ~~Introduction to parallel processing (RA) – 13/02/2025~~
- ~~Parallel processing: Basic data structures and communication (RA) – 20/02/2025~~
- ~~Paradigms for parallel programs (RA) – 27/02/2025~~
- ~~Analytical modelling of parallel systems and applications (RA) – 06/03/2025~~

High performance programming, hardware architecture verification

- ~~Practical aspects of parallel programming (SNE) – 13/03/2025~~
- ~~Distributed memory parallel programming (RA) – 20/03/2025~~
- ~~Shared memory parallel programming (SNE) – 27/03/2025~~
- ~~GPU programming & Vectorization (SNE) – 03/04/2025~~
- **Testing and verification (RA) – 24/04/2025**

Workshop: Design and implementation of high-performance programs – 01/05/2025

- ~~Problems released – 27/03/2025 04/04/2025~~
- **Report submission – 08/05/2025**

Contents

1. Introduction
2. Introduction to testing
3. Unit testing
4. Performance testing
5. Summary and exercises
6. Examination

1.Introduction

The purpose of testing and verification

Will my code give the correct answer consistently?

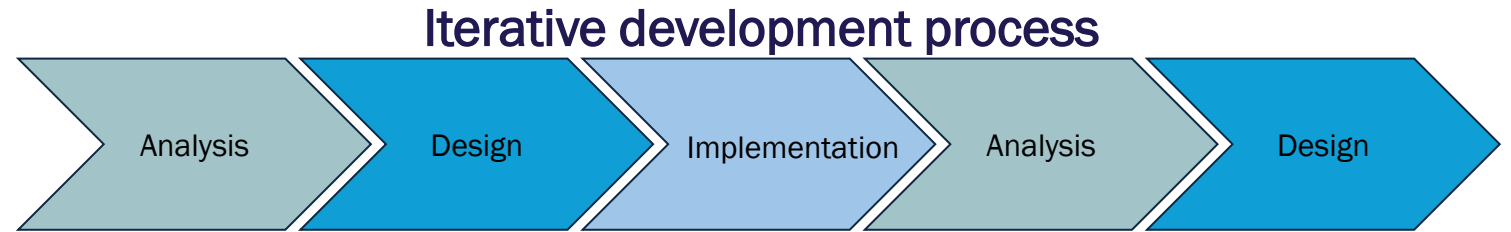
- Analysis
- Design
- Implementation

Will my code give the correct answer when needed?

- Design
- Implementation

Is my code resource-efficient?

- Design
- Implementation
 - CPU usage
 - GPU usage
 - Memory: registers, cache, RAM



Trade-off: parallel computing usually results in a higher memory usage

Complexity of parallel programming

Complete programs may need to incorporate multiple parallel algorithms

Involving different data structures, partitioning techniques, communication...

Modular design techniques: Encapsulate complex and changeable aspects inside separate program components or **modules**

Complete programs are a collection of (hopefully) well connected modules



KISS: Keep It Simple Stupid

A simple solution is better than a complex one, even if the solution looks stupid.

2. Introduction to testing

Passive/Static vs. dynamic testing

Passive/Static testing: Code review

Dynamic testing: unit tests, integration tests, acceptance tests

Coverage: all statements must be reached by tests

Worst-case test

You should use automated test tools

Passive testing: code review

Meeting or single reviewer

Useful to spot errors (walkthrough)

Coding style uniformity and **documentation** # This should encourage documenting codes

The team can become familiar with other modules

Code review document or issues on bug-tracking platforms

Testing

Unit testing: each individual module, function, and method **separately**

Pitfall: Functions consuming too much memory may be fast in unit testing

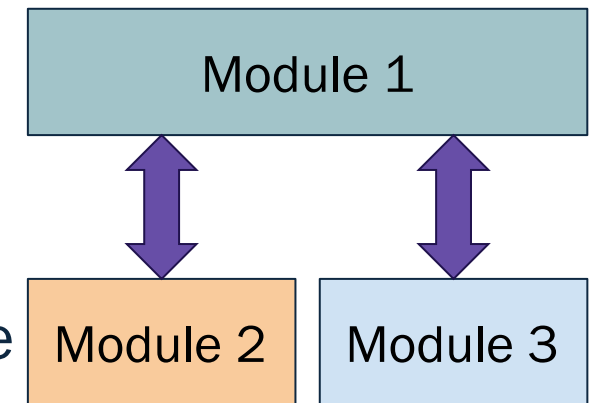
Integration testing: modules working together

Performance testing: speed. Do modifications lead to speedup?

Identify and isolate hot spots. Focus your optimization here

Varying clock counts: CPUs change the clock frequency depending on the workload

1. Warm up the CPU: Have you noticed the first run is the slowest one?
2. Disable power saving options



Testing

Coverage: all statements must be reached by tests

Worst-case test

International Software Testing Qualifications Board (ISTQB)

Test condition: A testable aspect of a component or system identified as a basis for testing.

Test case: A set of preconditions, inputs, actions (where applicable), expected results and postconditions, developed based on test conditions.

Many choices for automated testing tools: JUnit, Eclipse, ..., Google, Python



Test-driven development (TDD)

A software development process or methodology

Defines the order, control, and evaluation of the basic tasks involved in creating software

What? programming practice

How? producing automated tests for production code,
and using that process to drive design and programming

Goal: ‘clean code that works.’

Produce exactly as much code as will enable that tests to pass

*Janzen, David, and H. Saiedian. 2005. “Test-Driven Development Concepts, Taxonomy, and Future Direction.” Computer 38 (9): 43–50. <https://doi.org/10.1109/MC.2005.314>.

3. Unit testing

Unit testing

White-box: you know the implementation details.

Check if a single method complies with the specified behavior.

Path testing: does the program go through the correct statements?

Are things executed in the correct order? Compare with sequence and interaction diagrams

Data flow testing: do variables have the expected values at a given point in the code?

Assignments and references. Remember to use `new_var = var.copy()`

Race conditions can occur when multiple threads are running.

Fault injection: how does the method handle invalid inputs?

Condition/decision testing: all possible outcomes have been tested at least once

Test cases and assertions in Python

Test case: Checks for a specific response to a particular set of inputs.

Assertions are used to make sure some conditions remain True: **test cases**

- Pre-conditions: check for valid inputs
- Post-conditions: check for valid results

Otherwise, an **AssertionError** is raised and the program execution stops

...so, do not use them for branching

Syntax:

```
assert condition, "Assertion error message"
```


Common assertion types and **syntax**

Comparison: Is a value correct or valid?

- `a == b`
- `a > b`
- `0 < a`

Membership: Is an element in an array, list, or set?

- `a in A_list`
- `a not in A_list`

Identity: Are two variables the same?

- `x is y`
- `x is not y`

Type check: Is a variable of a certain class?

- `assert isinstance(x, int)`
- `assert isinstance(x, float)`

To make complex assertions

Create combinations using **and** / **or**

Use with functions that return a **bool**

Example:

```
assert np.allclose(a,b)
```

4. Performance testing

Performance testing

Goes beyond the traditional metrics for complexity and work

Is the average execution time sufficient to give performance guarantees?

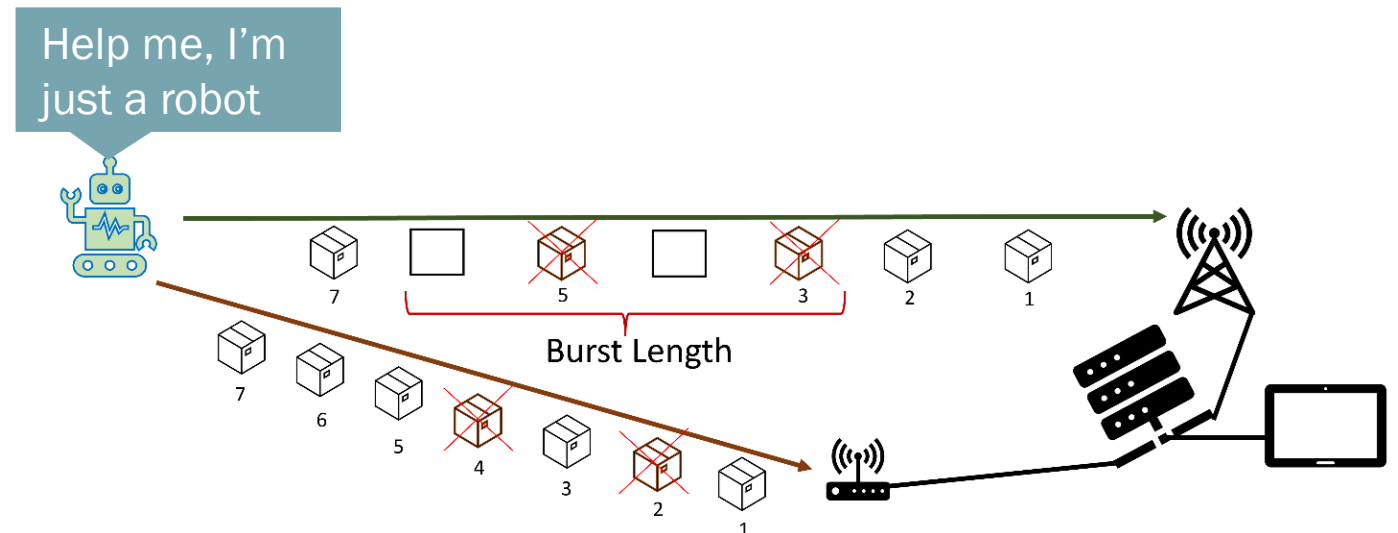
Only if there are no specific deadlines to meet

What happens when the execution time misses a deadline?

To be answered by the specific application: video, gaming, control, ...

What is the reliability of your code meeting a given deadline?

Fraction of times that execution time meets the deadline



Optimization objective

What my grandma thinks I do:

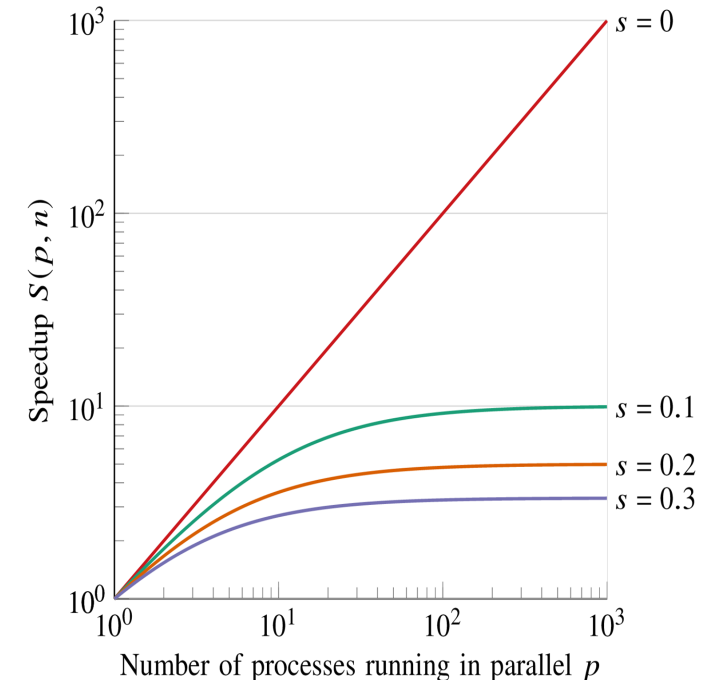
minimize Execution time with p parallel processes $T_p(n)$

subject to Work to be done (sequential and parallel)

Number of processors

Clock frequency of processor

Parallelisation overhead



Optimization objective

What I should do **but I usually don't**:

minimize f (Execution time, memory usage, CPU usage)

subject to Work to be done (sequential and parallel)

Number of processors

Clock frequency of processor

Memory latency/capacity

Parallelisation overhead

Deadline

The result is correct

Profilers

Complex programs are a large collection of tasks

Optimizing for performance takes time

We need to focus on the large time consumers in our programs

We are looking for:

- Unreasonably large number of calls
- Unreasonably large execution times

Sometimes due to the complexity of the algorithm but sometimes due to a coding error

So far, we have timed the execution of each function with a decorator `@timer`

Profilers give more information. Caution: they usually give the average execution time

Python profilers

cProfile and profile (manual: <https://docs.python.org/3/library/profile.html>)

Can be run outside of Python

```
python -m cProfile [-o output_file] [-s sort_order] (-m module | myscript.py)
```

Output

- **ncalls:** Number of times that a function is called in the form total/primitive calls
- **tottime:** Total time spent in the given function **excluding** subfunctions
- **cumtime:** Total time spent in the given function **including** its subfunctions
- **percall:** quotient of cumtime divided by primitive calls

cProfile options

-o writes the results to a file instead of printing them on screen

-s specifies the stats to sort the list after. Cannot be combined with -o

Example

```
python -m cProfile -s cumtime -m module
```

A good option: Print the profiling results to a file and use the pstats module

```
python3 -m cProfile -o profile_results.stats -m module
```

```
python3 -m pstats profile_results.stats
```


Pstats to make the stats readable

You can print the profiling results to a file and use the pstats module to sort them

Saving to a file

```
python3 -m cProfile -o profile_results.stats -m module
```

Opening the profiling results

```
python3 -m pstats profile_results.stats
```

Sorting according to cumtime (descending order)

```
profile_results.stats% sort cumtime
```

Print the 10 first results

```
profile_results.stats% stats 10
```

You can always ask for “help”

5. Summary and exercises

Summary of the practical part of the course

Not all the tasks must be completed sequentially

Vectorization: use greatly optimized operations to solve specific SIMD problems

Parallel programming: Distribute the work among processing elements

- **Multithreading:** Global memory, so data consistency must be enforced
- **Multiprocessing:** Mostly local memory with a few instructions to share data globally
- **MPI:** Distributed memory, so all data must be passed through an interface
- **GPU programming:** Many processing elements, low control capabilities. You move the data

Testing and verification: optimizations must result in a relevant speedup and don't introduce errors

Exercises

- 1. Test referencing vs. copying arrays/lists in Python.** That is, make Python array/list `x`. Then, make `y = x` and `z = x.copy()`. Are `x`, `y`, and `z` equal? Are `y` and `z` the same as `x`?
- 2. Matrix structure:** To work correctly, a module requires an upper triangular matrix to work correctly. Make an assertion to test that this is actually passed to the module.
- 3.** Make an assertion to ensure that all the rows in a matrix sum to 1. Then, make a code that ensures this is true.
- 4. Rounding errors:** Define `a = 7.33`, `c = a**b`, and `d = c**(1/b)`. How large can `b` be for `assert(a==d)` to pass without `c` being “too large”?
- 5. Profiling inside code:** Running `cProfile` on a large code generates a lot of “garbage,” so it might be a good idea to profile only the functions of interest inside the code. Modify the code for matrix multiplication with `numba` (in the moodle folder) to profile the functions inside the code.

Examination

- Examination is oral
 - Goal is to evaluate your mastery of the **key concepts covered in the course** and **the study regulation**.
- Each student is allocated a total of 20 minutes (including 15-16 mins exam and 4-5 mins for voting and grade announcement)
- The assessment is based on the 7-point grading system.
- If submitted, the first part of the exam will include questions to evaluate understanding of the higher performance programming concepts applied to solve the workshop problems. Submission of a workshop report is recommended **but not a requirement for the exam**.
- Allowed material: **Workshop report AND a summary sheet with 5 pages maximum**.
- The examination will cover the entire course.

That's all for the course